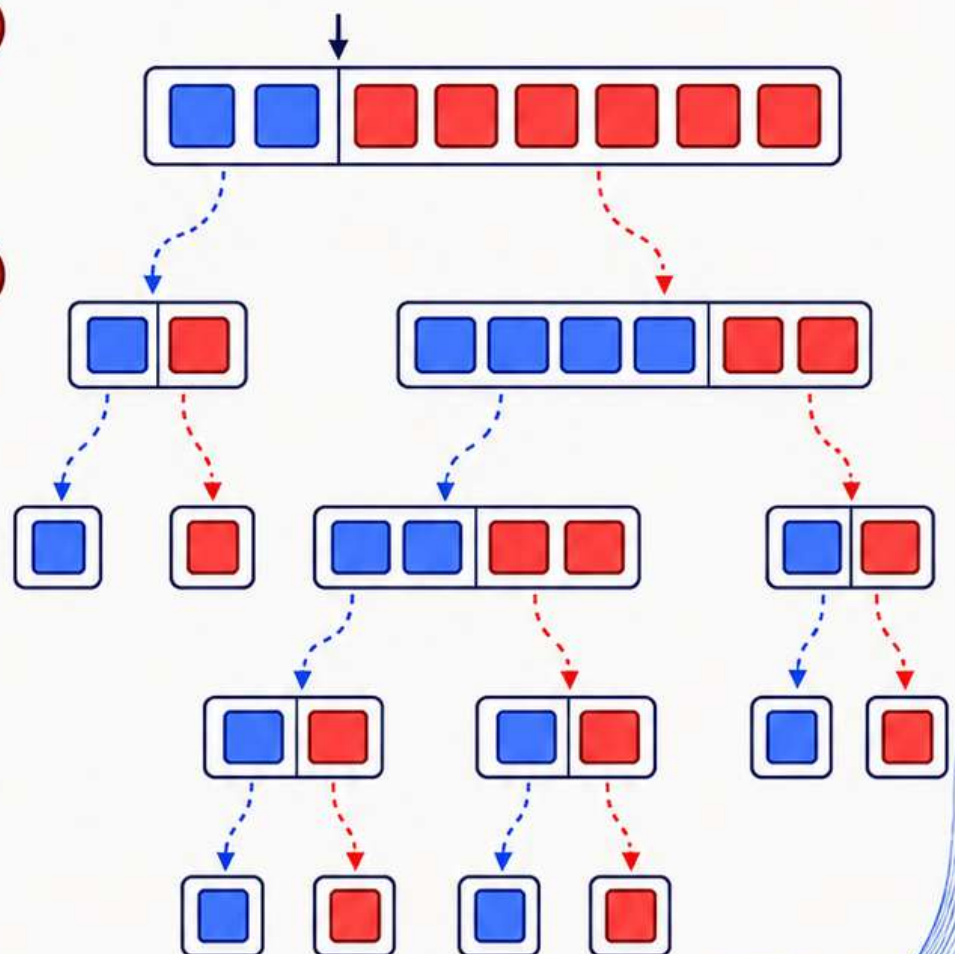
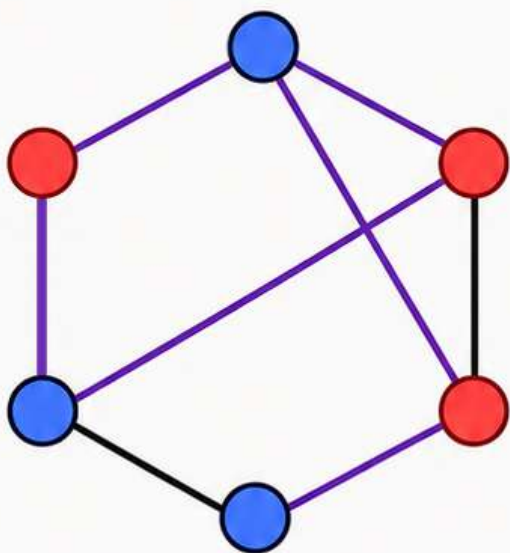


Randomised Algorithms

Sahil M Deo



Contents

1	Introduction	5
1.1	What is a Randomised Algorithm?	5
1.2	Implementation of Randomness in C++	5
1.3	Problems	8
2	Algorithmic Performance	9
2.1	Worst Case Analysis	9
2.2	Average Case Analysis	9
2.3	Time Complexity Analysis	10
2.4	Problems	12
3	Complexity Classes	13
3.1	Decision Problems	13
3.2	P	13
3.3	NP	14
3.4	Polynomial-Time Reductions	14
3.5	NP-Complete	14
3.6	RP	15
3.7	ZPP	15
3.8	BPP	15
3.9	Problems	16
4	The Classics	17
4.1	QuickSort	17
4.1.1	The Algorithm	17
4.1.2	Worst Case Runtime	17
4.1.3	Expected Runtime	18
4.2	QuickSelect	19
4.2.1	The Algorithm	19
4.2.2	Expected Runtime	20
4.2.3	Expected Recursion Depth	20
4.3	Problems	22
5	Hashing	23
5.1	What even is Hashing?	23
5.2	Polynomial Multiplication	23
5.3	Matrix Multiplication	24
5.3.1	Freivalds' Algorithm	24
5.4	A Matrix Problem	25
5.4.1	Deterministic Method	26
5.4.2	Hashing Method	26

5.5	A Sliding Window Problem	28
5.6	Problems	30
6	Data Structures	31
6.1	Skip Lists	31
6.1.1	The Structure	33
6.1.2	Expected Height	33
6.1.3	Search	34
6.1.4	Insertion	36
6.2	Treaps	37
6.2.1	Binary Trees	37
6.2.2	Tree Traversal	38
6.2.3	Binary Search Trees	38
6.2.4	Balancing	42
6.2.5	Rotations	43
6.2.6	Min-Heaps	44
6.2.7	Treaps	47
6.3	Hash Tables	50
6.3.1	The Structure	50
6.3.2	Insertion	51
6.3.3	Search	52
6.3.4	Deletion	52
6.4	Bloom Filters	52
6.4.1	The Structure	53
6.4.2	Insertion	53
6.4.3	Search	53
6.4.4	Utility of Bloom Filters	53
6.5	Cuckoo Hashing	54
6.5.1	The Structure	54
6.5.2	Insertion	54
6.5.3	Search	55
6.6	Problems	56
7	Number-Theoretic Algorithms	57
7.1	Introduction	57
7.2	Fermat Test	57
7.2.1	Fermat's Little Theorem	58
7.2.2	The Algorithm	58
7.2.3	Error Probability	58
7.3	Miller–Rabin Test	59
7.3.1	A Key Property	59
7.3.2	The Algorithm	60
7.3.3	Error Probability	60
7.3.4	Performance	61
7.4	Pollard's Rho Algorithm	61
7.4.1	Floyd's Cycle-Detection Algorithm	62
7.4.2	The Algorithm	64
7.4.3	Performance	65
7.5	Problems	68

8	Probabilistic Method	69
8.1	Dominating Set	69
8.2	Bipartite Subgraph	71
8.2.1	Expected Number of Bichromatic Edges	71
8.2.2	A Simple Randomised Algorithm	72
8.2.3	Derandomisation	72
8.3	Array Construction	74
9	Online Algorithms	77
9.1	Dating	77
9.2	Load Balancing	80
9.3	Distinct Elements	82
9.3.1	Flajolet–Martin Method	82
9.3.2	HyperLogLog Method	84
10	Heuristic Algorithms	85
10.1	Dominating Set Revisited	85
10.2	Independent Set	87
10.2.1	Random Permutation	87
10.2.2	Local Decision	88
10.3	Bipartite Subgraph Revisited	90
10.3.1	Self-Correction	90
	Appendices	91
A	Probability and Expectation	92
A.1	Linearity of Expectation	92
A.2	Tail Sum Formula	92
A.3	Union Bound	92
A.4	Independence of Expectation	92
A.5	Variance	92
A.6	Indicator Variables	93
A.7	Law of Total Expectation	93
A.8	Wald’s Equation	93
B	Inequalities	95
B.1	Exponential Inequality	95
B.2	Jensen’s Inequality	95
B.3	Markov’s Inequality	96
C	Convergence and Limit Theorems	97
C.1	Identical and Independently Distributed Random Variables	97
C.2	Almost Surely	97
C.3	Almost Never	97
C.4	$\lim_{n \rightarrow \infty} X_n = Y$	98
C.5	Convergence in Probability	98
C.6	The Weak Law of Large Numbers	98
C.7	Almost Sure Convergence	99
C.8	A_n infinitely often	99
C.9	The Borel–Cantelli Lemmas	99
C.10	The Strong Law of Large Numbers	100
C.11	Convergence in Distribution	100

C.12 The Central Limit Theorem	101
D Discrete Mathematics	102
D.1 Pigeonhole Principle	102
D.2 Relations	102
D.2.1 Partial Orders	102
D.2.2 Equivalence Relations	103
D.3 Graph Theory	103
E Linear Algebra	104
E.1 Vector Spaces	104
E.2 Subspaces	104
E.3 Dimension of a Subspace	104
E.4 The Nullspace	105
E.5 Nullity and Rank	105
E.6 The Rank–Nullity Theorem	105

Chapter 3

Complexity Classes

What is the best time complexity for a problem? When can we hope to come up with a *fast* algorithm? This is exactly the point of defining complexity classes are about. While the classes themselves are defined rigorously, there are some very important¹ things which we have not yet proved – they remain open problems but in practice we have conjectures which we believe to be true for now...

3.1 Decision Problems

Complexity classes are traditionally defined using *decision problems*. A decision problem is a computational problem whose answer is either YES or NO.

- Given a graph G , is G connected?
- Given an integer m , is m prime?
- Given a graph G and an integer k , does G contain a clique² of size at least k ?

Although not all computational tasks are naturally decision problems, most can be reformulated as one. For example, the largest clique in a graph can be reformulated³ as "is there a clique of size at least k in G ?".

3.2 P

The class **P** consists of all decision problems that can be solved by a deterministic algorithm in at most polynomial time. Let Q be a decision problem and $T(Q)$ be the best time complexity⁴ of a deterministic algorithm for Q . Then $T(Q) \leq \Theta(n^c)$ for some constant $c \geq 1$.

$$\mathbf{P} = \bigcup_{c \geq 1} \{Q : T(Q) \leq \Theta(n^c)\}$$

Decision Problem(Q)	Algorithm	Input	Input Size	Runtime (T(Q))
Primality Test	AKS	Integer	$m = \Theta(\log_2 n)$	$\Theta(m^{12})$
Graph Connectivity	BFS	Adjacency Matrix	$n = V \times V $	$\Theta(n)$
Interval Scheduling	Sorting and Greedy	Start-End times	nm	$\Theta(nm \log n)$

¹Such as the classic question of $\mathbf{P} \neq \mathbf{NP}$...

²Refer Appendix D.3

³Some of these reformulations are called *polynomial time reductions* which will be properly discussed later.

⁴This time complexity is in terms of the input length and not the input number itself – a number n when taken as input is represented by a bit string of size $m = \Theta(\log n)$. So an algorithm which takes $O(n)$ time will be considered as an exponential time algorithm since $\Theta(n) = \Theta(2^m)$. This is why the normal method of trying all numbers from 1 to $\sqrt{n}$ will not prove that primality testing of a number is in P .

3.3 NP

The class NP consists of all decision problems whose answer when **YES** can be verified in polynomial time. The algorithm which solved the problem can output a *certificate* along with **YES** – this certificate is information which can be used to verify in polynomial time that the answer is indeed **YES**.

Decision Problem (Q)	Certificate	Verification Time
Clique of size at least k	Set of k vertices	$\Theta(k^2)$
Graph Coloring of k colors	Vertex : Color mapping	$\Theta(V + E)$

An obvious conclusion from the definition of **NP** is that $P \subseteq NP$ because any problem that can be solved in polynomial time can also be verified in polynomial time by giving the problem itself as a certificate.

NP_{co}

The class NP_{co} contains complements of problems in **NP**. This means that the answer **NO** has a certificate verifiable in polynomial time. For example, the complement of the Clique problem asks whether every clique in the graph has size at most $k-1$. The answer **NO** can be accompanied by a certificate which is the set of vertices forming a clique of size at least k .

An obvious conclusion is that any algorithm that solves an **NP** problem also immediately solves the corresponding NP_{co} problem by just flipping the answer from **YES** to **NO** and vice versa but it may not be able to provide a certificate in polynomial time.

3.4 Polynomial-Time Reductions

To compare the difficulty of computational problems, we use the idea of *polynomial-time reductions*. A problem A is polynomial-time reducible to a problem B , then

$$A \leq_p B$$

if we can convert A to B in polynomial time. This means that if we can solve B in polynomial time, then we can also solve A in polynomial time. The notation suggests B is harder than A and you might find this confusing because we "reduced" A to B so shouldn't B be easier? Think of a reduction as a one-way procedure that converts A to a *particular* case of B . Solving the general case of a problem is at least as hard as solving the particular case. That is why B is at least as hard as A under polynomial time reduction.

The operator $<_p$ is a partial order⁵ which is defined as $A <_p B$ iff $A \leq_p B$ but not $B \leq_p A$. We can also define $=_p$ as $A =_p B$ iff $A \leq_p B$ and $B \leq_p A$. This can be used to induce equivalence classes⁶ within the complexity classes like **NP – complete** which is the "biggest" equivalence class in **NP**.

3.5 NP-Complete

NP – complete consists of those **NP** problems which are at least as hard as *every* problem in **NP**. For an L in **NP – complete**, every problem in **NP** can be polynomially reduced to L .

$$\mathbf{NP - complete} = \{L : Q \leq_p L \quad \forall Q \in \mathbf{NP}\}$$

⁵Refer Appendix D.2.1

⁶Refer Appendix D.2.2

3.6 RP

A decision problem belongs to **RP** if there exists a randomised polynomial-time algorithm with input size n satisfying the following properties:

- If the correct answer is **NO**, the algorithm always outputs **NO**.
- If the correct answer is **YES**, the algorithm outputs **YES** with probability at least $\frac{1}{n^c}$, $c > 0$.

Consider an algorithm as mentioned above – it outputs **YES** in the case where correct answer is **YES** with probability $p \geq \frac{1}{n^c}$. Run it k times and answer **YES** if it occurs even once in the k runs otherwise answer **NO**. This new algorithm has the same properties as the original algorithm except p is replaced by $1 - (1 - p)^k$. So if an algorithm has $p \geq \frac{1}{n^c}$, then for $k = n^c$ the probability will be more than $1 - \frac{1}{e}$.

$$\left(1 - \frac{1}{n^c}\right)^{n^c} \leq \frac{1}{e} \implies 1 - \left(1 - \frac{1}{n^c}\right)^{n^c} \geq 1 - \frac{1}{e}$$

As long as k is polynomial in the input size, the overall algorithm is also polynomial time. The point of having a lower bound $\frac{1}{n^c}$ is that we should be able to achieve any desired correctness probability by repeating the algorithm a polynomial number of times – this may not be possible if our criterion is something like $\frac{1}{2^n}$. Once we get a lower bound of $1 - \frac{1}{e}$, we can do a constant number of repetitions (still keeping the algorithm overall polynomial) to achieve a probability which is as close to 1 as we desire.

RP_{co}

A problem belongs to **RP_{co}** if there exists a randomised polynomial-time algorithm satisfying the following properties:

- If the correct answer is **YES**, the algorithm always outputs **YES**.
- If the correct answer is **NO**, the algorithm outputs **NO** with probability at least $\frac{1}{2}$.

To clarify, when we say randomised polynomial time algorithm, we mean an algorithm which makes random choices but whose time complexity is polynomial in the input size. The relevant time complexity here is the actual worst-case time complexity and **not** the expected time complexity.

3.7 ZPP

A decision problem belongs to **ZPP** if there exists a randomized algorithm that has the following properties:

- It outputs the correct answer with probability 1.
- Expected time complexity is polynomial in the input size.

ZPP contains all those problems which can be solved correctly and *efficiently in expectation*.

3.8 BPP

A decision problem belongs to **BPP** if there exists a randomized polynomial-time algorithm whose probability of producing the correct answer is at least $\frac{1}{n^c}$, where $c > 0$.

3.9 Problems

- 1 Solving an **NP – complete** problem in polynomial time is considered extremely difficult (if not impossible). Let C be the event that a single **NP – complete** problem is solved in polynomial time.

(a) Show that $C \implies \mathbf{P} = \mathbf{NP} = \mathbf{NP} - \text{complete}$

(b) Show that $\mathbf{P} \subseteq (\mathbf{NP}_{\text{co}} \cap \mathbf{NP}) \subseteq \mathbf{NP}$ and thus show $C \implies \mathbf{NP} = \mathbf{NP}_{\text{co}}$

- 2 Modern cryptography relies on the existence of one-way functions. One-way functions are functions which are computable in polynomial time but not invertible in polynomial time. Consider the function $F : \{0, 1\}^n \mapsto \{0, 1\}^m$.

(a) Show that if F is invertible, $m \geq n$.

(b) Using the above result, show that if $\mathbf{P} = \mathbf{NP}$ then F can not be a one-way function by reformulating the invertibility problem as a decision problem. *Hint: Construct the inverse bit-by-bit.*

- 4 Consider an algorithm **ALGO** which solves a decision problem $Q \in \mathbf{ZPP}$ with runtime T whose expected value is E such that $E \leq n^c$ where $c > 0$. Now consider the algorithm **TIMED_ALGO**:

```

TIMED_ALGO
{
    run ALGO with time limit 2*E
    if time limit exceeded
        return NO
    else
        return answer of ALGO
}

```

Let p be the probability that **TIMED_ALGO** gives the correct answer.

(a) Show that $p \geq \Pr(T \leq 2E)$

(b) Use the markov inequality to show that $\Pr(T > 2E) \leq \frac{1}{2}$

(c) Use the answers to (a) and (b) to show that the problem is solved by **TIMED_ALGO** with probability at least $\frac{1}{2}$.

(d) Using the result from (c), show that $Q \in \mathbf{BPP}$. Subsequently conclude that this proof works for a general $Q \in \mathbf{ZPP}$, thus proving that $\mathbf{ZPP} \subseteq \mathbf{BPP}$

(e) Use a similar setup to show $\mathbf{ZPP} \subseteq \mathbf{RP} \cap \mathbf{RP}_{\text{co}}$

(f) Use a **RP** algorithm and a **RP_{co}** algorithm to show that $\mathbf{RP} \cap \mathbf{RP}_{\text{co}} \subseteq \mathbf{ZPP}$, thus proving $\mathbf{ZPP} = \mathbf{RP} \cap \mathbf{RP}_{\text{co}}$